

.NET cryptography model

02/13/2024

.NET provides implementations of many standard cryptographic algorithms.

Object inheritance

The .NET cryptography system implements an extensible pattern of derived class inheritance. The hierarchy is as follows:

- Algorithm type class, such as [SymmetricAlgorithm](#), [AsymmetricAlgorithm](#), or [HashAlgorithm](#). This level is abstract.
- Algorithm class that inherits from an algorithm type class, for example, [Aes](#), [RSA](#), or [ECDiffieHellman](#). This level is abstract.
- Implementation of an algorithm class that inherits from an algorithm class, for example, [AesManaged](#), [RC2CryptoServiceProvider](#), or [ECDiffieHellmanCng](#). This level is fully implemented.

This pattern of derived classes lets you add a new algorithm or a new implementation of an existing algorithm. For example, to create a new public-key algorithm, you would inherit from the [AsymmetricAlgorithm](#) class. To create a new implementation of a specific algorithm, you would create a non-abstract derived class of that algorithm.

Going forward, this model of inheritance is not used for new kinds of primitives like [AesGcm](#) or [Shake128](#). These algorithms are `sealed`. If you need an extensibility pattern or abstraction over these types, the implementation of the abstraction is the responsibility of the developer.

One-shot APIs

Starting in .NET 5, simpler APIs were introduced for hashing and HMAC. While slightly less flexible, these *one-shot* APIs:

- Are easier to use (and less prone to misuse)
- Reduce allocations or are allocation-free
- Are thread safe
- Use the best available implementation for the platform

The hashing and HMAC primitives expose a one-shot API through a static `HashData` method on the type, such as [SHA256.HashData](#). The static APIs offer no built-in extensibility mechanism. If you're implementing your own algorithms, it's recommended to also offer similar static APIs of the algorithm.

The [RandomNumberGenerator](#) class also offers static methods for creating or filling buffers with cryptographic random data. These methods always use the system's cryptographically secure pseudorandom number generator (CSPRNG).

How algorithms are implemented in .NET

As an example of the different implementations available for an algorithm, consider symmetric algorithms. The base for all symmetric algorithms is [SymmetricAlgorithm](#), which is inherited by [Aes](#), [TripleDES](#), and others that are no longer recommended.

[Aes](#) is inherited by [AesCryptoServiceProvider](#), [AesCng](#), and [AesManaged](#).

In .NET Framework on Windows:

- `*CryptoServiceProvider` algorithm classes, such as [AesCryptoServiceProvider](#), are wrappers around the Windows Cryptography API (CAPI) implementation of an algorithm.
- `*Cng` algorithm classes, such as [ECDiffieHellmanCng](#), are wrappers around the Windows Cryptography Next Generation (CNG) implementation.
- `*Managed` classes, such as [AesManaged](#), are written entirely in managed code. `*Managed` implementations are not certified by the Federal Information Processing Standards (FIPS), and may be slower than the `*CryptoServiceProvider` and `*Cng` wrapper classes.

In .NET Core and .NET 5 and later versions, all implementation classes (`*CryptoServiceProvider`, `*Managed`, and `*Cng`) are wrappers for the operating system (OS) algorithms. If the OS algorithms are FIPS-certified, then .NET uses FIPS-certified algorithms. For more information, see [Cross-Platform Cryptography](#).

In most cases, you don't need to directly reference an algorithm implementation class, such as `AesCryptoServiceProvider`. The methods and properties you typically need are on the base algorithm class, such as `Aes`. Create an instance of a default implementation class by using a factory method on the base algorithm class, and refer to the base algorithm class. For example, see the highlighted line of code in the following example:

```
C#
```

```
using System.Security.Cryptography;

try
{
    using (FileStream fileStream = new("TestData.txt", FileMode.OpenOrCreate))
    {
        using (Aes aes = Aes.Create())
        {
            byte[] key =
            {
                0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08,
                0x09, 0x10, 0x11, 0x12, 0x13, 0x14, 0x15, 0x16
            };
            aes.Key = key;

            byte[] iv = aes.IV;
            fileStream.Write(iv, 0, iv.Length);

            using (CryptoStream cryptoStream = new(
                fileStream,
                aes.CreateEncryptor(),
                CryptoStreamMode.Write))
            {
                // By default, the StreamWriter uses UTF-8 encoding.
                // To change the text encoding, pass the desired encoding as the
                second parameter.
                // For example, new StreamWriter(cryptoStream, Encoding.Unicode).
                using (StreamWriter encryptWriter = new(cryptoStream))
                {
                    encryptWriter.WriteLine("Hello World!");
                }
            }
        }
    }

    Console.WriteLine("The file was encrypted.");
}
catch (Exception ex)
{
    Console.WriteLine($"The encryption failed. {ex}");
}
```

Choose an algorithm

You can select an algorithm for different reasons: for example, for data integrity, for data privacy, or to generate a key. Symmetric and hash algorithms are intended for protecting data for either

integrity reasons (protect from change) or privacy reasons (protect from viewing). Hash algorithms are used primarily for data integrity.

Here is a list of recommended algorithms by application:

- Data privacy:
 - [Aes](#)
- Data integrity:
 - [HMACSHA256](#)
 - [HMACSHA512](#)
- Digital signature:
 - [ECDsa](#)
 - [RSA](#)
- Key exchange:
 - [ECDiffieHellman](#)
 - [RSA](#)
- Random number generation:
 - [RandomNumberGenerator.GetBytes](#)
 - [RandomNumberGenerator.Fill](#)
- Generating a key from a password:
 - [Rfc2898DeriveBytes.Pbkdf2](#)

See also

- [Cryptographic Services](#)
- [Cross-Platform Cryptography](#)
- [ASP.NET Core Data Protection](#)